Claude Code for research

Bahlo lab meeting

Stuart Lee

2026-05-12

Three items today

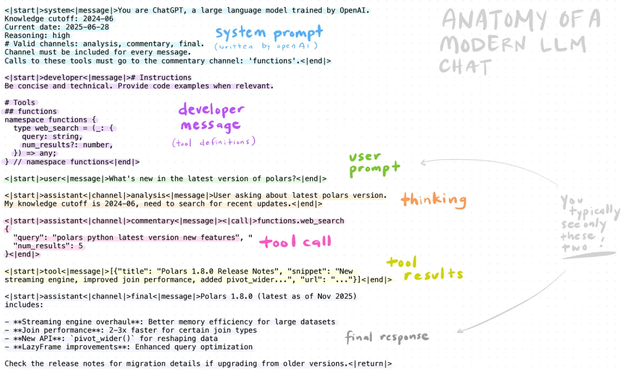
1. How it works.
2. How I use it.
3. What to watch for.

Then group activities.

Section 1

How it works

An LLM chat is one long prompt that grows with every turn



A token is a subword piece — about three-quarters of a word, four characters on average. The context window is measured in tokens (Opus 4.7 on the Team plan: 1M).

Confident wrong answers are a feature of the architecture

Without tools, it generates fluent text — not retrieved facts.

Fluent and wrong sound the same.

The context window is the model's only memory

Context window (this turn)

- file you opened
- error you pasted
- tool result

= the model's only memory

Not in the buffer:

previous session

files you didn't open

tool calls in other turns

If it "forgot" — it never had it.

Grounding turns guesses into answers we can check

Show the actual file, the actual error, the actual data.

What's in the buffer matters more than which model you chose.

Section 2

From web to the command line

In a web chat, you are the bridge between model and reality

You read code in. You copy answers out. You re-paste errors.

Fine for one-off questions. Does not scale to a workflow.

Claude Code, concretely

A coding-agent CLI from Anthropic. Point it at a directory; it gets your files, your shell, git, and the web.

- Runs in your terminal, in VS Code, or at `claude.ai/code`.
- Calls Anthropic's API for each model turn.
- Sees what you point it at.

Models: Sonnet 4.6 for everyday coding; **Opus 4.7** for harder reasoning and longer context.

Not the only harness — **Codex** (OpenAI), **Gemini CLI**, **OpenCode** do similar work. Lessons here transfer.

An agent reads files, runs commands, loops on results

1. **Think**
2. **Act** — read, run, call.
3. **Observe**
4. Repeat.

Can do unattended

Read files, run shells, edit, web, MCP.

Cannot do

Know your goal unless you say it.

More autonomy = more chances to be wrong unnoticed.

MCP servers turn Claude into a research client

MCP (Model Context Protocol) — a shared interface that lets Claude call out to external services and data sources.

- **PubMed** — search, metadata, full text.
- **bioRxiv / medRxiv** — preprints by topic, author, funder.
- **Microsoft 365** — email, files, calendar.

Section 3

How I actually use it

TDD — test-driven development — gives agents a target you can check

`fraction = 0.5` should keep ranges with at least 50% overlap

Write the invariant first.

```r
# only x1 has >=50% of itself
# overlapping y
x <- GRanges("chr1",
  IRanges(c(1, 1, 100), width = c(10, 4, 5)))
y <- GRanges("chr1", IRanges(6, width = 10))

result <- filter_by_overlaps(
  x, y, fraction = 0.5
)
expect_identical(length(result), 1L)
expect_identical(start(result), 1L)
```

Route through `overlap_hits` when a fraction is set.

```r
# in filter_by_overlaps()
fraction <- resolve_fraction(fraction)

hits <- overlap_hits(
  x, y, type, minoverlap, maxgap,
  strand, fraction
)
x[hits$queryHits]
```

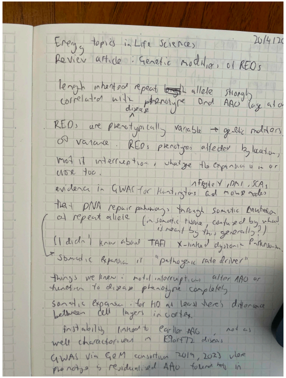
Claude with skills becomes an interactive lab book

Recurring workflows I wrote once as plain-text files in `.claude/commands/` and invoke with a slash.

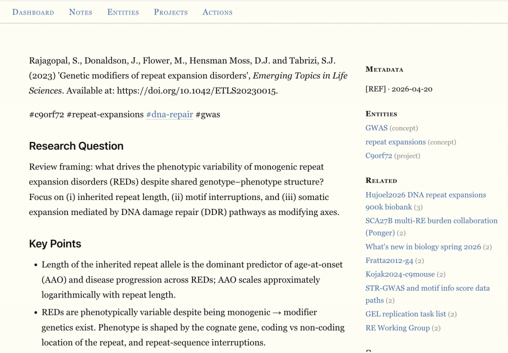
- `/capture` — thought, paper, fragment → structured note.
- `/meeting` — meeting transcript → actions, decisions, follow-ups.
- `/review` — week's end → summary, what's stuck, next week.

Not the author. The structure that lets me write more of my own thinking down.

A paper read becomes a queryable archive entry



Handwritten while reading.



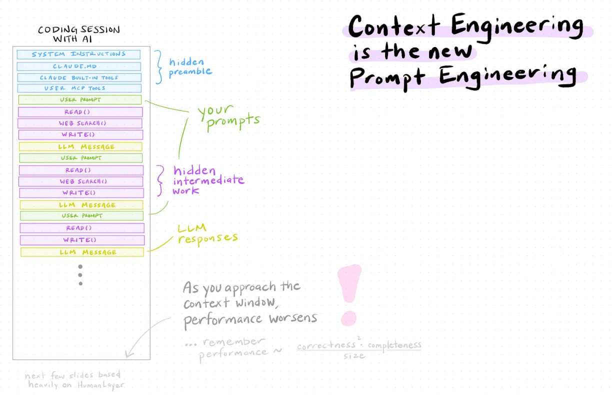
Linked, tagged, searchable from any future session.

One `/capture` per paper. My `notes/` directory is just markdown — Claude reads it the same way I do.

Section 4

Context management

Context is the bottleneck, not the model



What you put in beats how you phrase it.

CLAUDE.md and skills give Claude durable memory

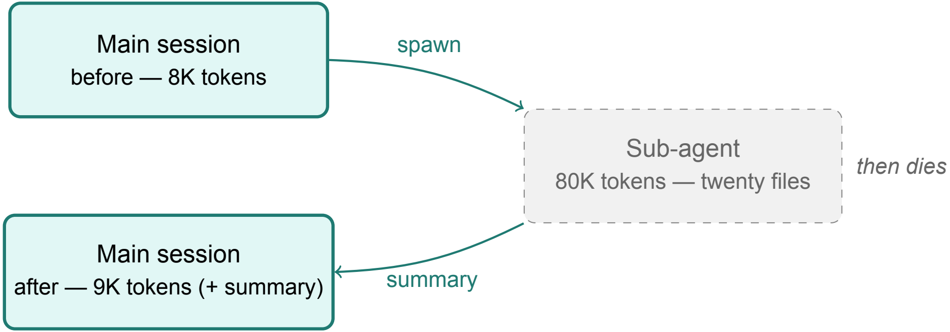
CLAUDE.md — always loaded.

- Conventions
- Domain vocabulary
- Project anti-patterns
- Pointers to source-of-truth files

Skills / slash commands — on demand.

- Reviewing a PR
- Bumping a package version
- Drafting a session log

Sub-agents delegates work away from the main session



Design docs survive sessions. **Drift** = repeating mistakes you've already corrected — restart when it starts.

Section 5

Verification

Git is Claude’s other memory

- **Rollback is cheap.** `git reset` undoes nonsense.
- **Memory is auditable.** ”What changed in the last three commits?” — concrete, not guessed.

Commit early, commit often.

Read the changes every time

- Code $\rightarrow$ run the test, quote the output.
- Data $\rightarrow$ row counts, dtypes, NA rates, before / after.
- Plot $\rightarrow$ render it. Look at it.
- Model $\rightarrow$ predicted vs. actual.

"It should work" is not verification.

Simulate with a known truth; check it's recovered

For any step that feeds a figure or a table:

1. Generate data with known ground truth.
2. Run the pipeline.
3. Confirm it recovers the truth within tolerance.

Substitute for cross-software replication. The simulation *is* the test fixture.

When it’s confused, re-ground — don’t argue

Vague prompt
“compare the two ancestries”

Plausible-looking wrong
“Northern vs Southern European…”
(silently swapped)

Grounded prompt
“compare gnomAD AFR vs EUR in `/data/.../finemap/`”

Verifiable correct
“AFR (n=8127) vs EUR (n= …)…”

Section 6

Pitfalls

Six ways this goes wrong

Premature action plan first	**Confabulated paths** verify by reading	**Sycophancy** it will agree if you push
Cost runaway set a budget	**Outsourced judgment** uses what you can’t verify	**Vibe coding** runs, but no one understands it

Section 7

Working with sensitive data

Data egress is the constraint

Fine

- Code-only HPC repos
- Pipeline + simulation code
- Manuscript drafts (code-only dirs)

Not fine without policy

- clinical data
- Mixed code/data directories
- ”It’s on the HPC” as control

Local models on the HPC unlock controlled-data work

The path forward

Run **open-weight models** (e.g. the **Qwen** family) on our HPC for work involving controlled-access data.

Until then: laptops ✓, code-only HPC ✓, controlled data off-limits.

Anthropic doesn't train on commercial-tier prompts; **Enterprise tier** adds zero-data-retention if we need it.

Section 8

Where to from here?

Now $\rightarrow$ next $\rightarrow$ ongoing

1. **Now** — work through `ACTIVITIES.md` together.
2. **Next week** — pick one activity, try it on your own work, bring back what broke.
3. **Ongoing** — regular lab catch-up to share tips, skills, and gotchas as we find them.